Building a universal game-playing agent infrastructure to optimize performance

Maria Manina
IM updates

13.01.2026

Research question: reminder

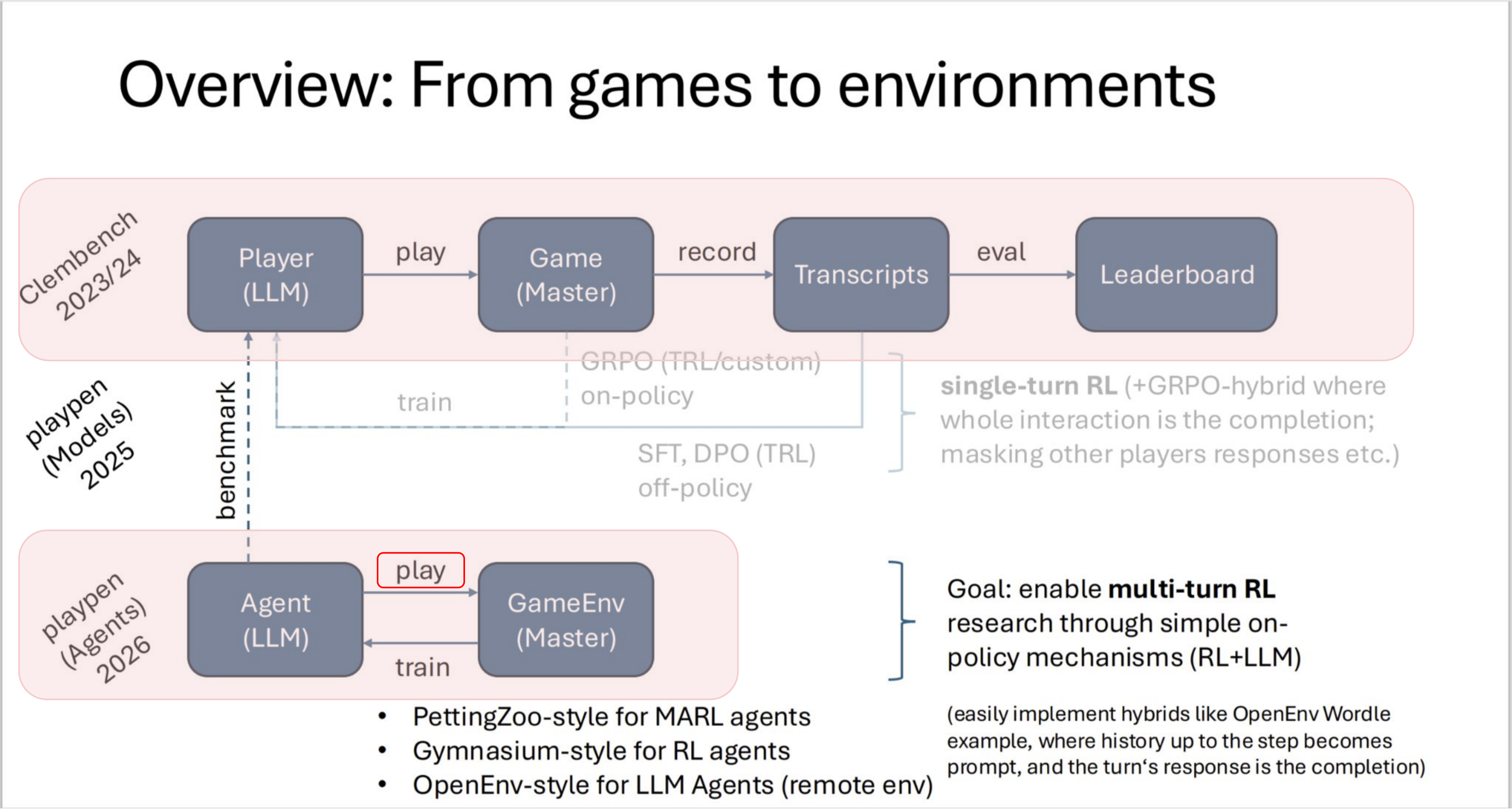
Goal: create a general dialogue game-playing agent that

- does not rely on game-specific solvers
- does not modify game-specific rules
- must adhere to the standard clembench game flow

Method: adding different modular components around vanilla LLMs (Reflect, ReAct, external databases access etc.)

Previously…

- ➔ a self-correction component was implemented inside the clemcore GM
- ➔ preliminary tests indicate that:
  - ◆ reflection + default temperature => lower performance
  - ◆ reflection + higher temperature => higher performance



Source: clemcore internal slides (Philipp)

clem: from games to agents

Although no training/learning is needed for my task, this setup can be used to:

1. Cleanly separate agent engineering from clemcore
2. Avoid coupling agents to GM-specific details
3. Test different agent architectures against each other

Acknowledgement: based on ideas and guidance from Prof. Schlangen and Philipp

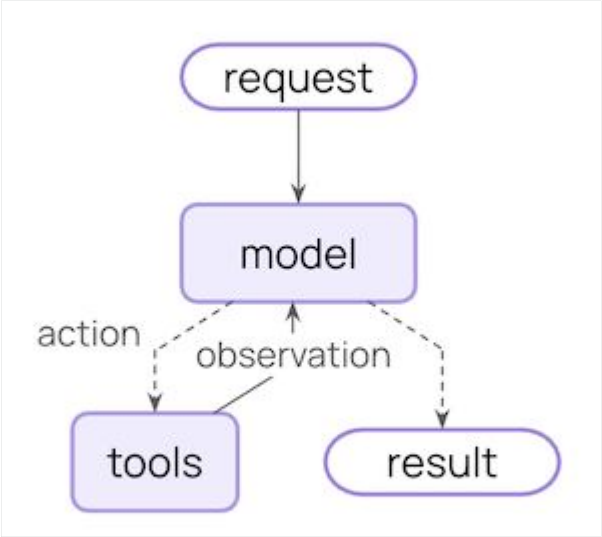
Using LangChain for agents

- langchain is a library that helps to quickly build custom agents
- it has templates for common use cases
- it has integrations across embedding models, tools, semantic search/processing platforms etc.
- more complex structures are available under Deep Agents (built on top of LangChain)

Using LangChain for agents

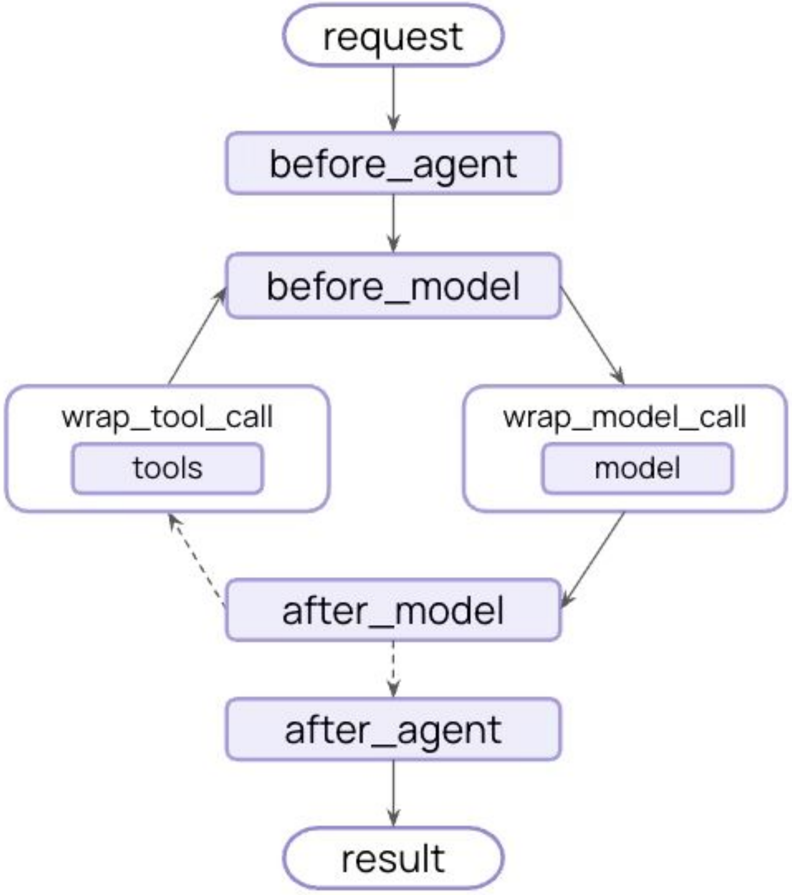
The core agent loop involves calling a model, letting it choose tools to execute, and then finishing when it calls no more tools.

Agents follow the ReAct ("Reasoning + Acting") pattern by default, alternating reasoning and tool use steps until they can deliver a final answer.



Using LangChain for agents

The agent loop can be more complicated:



Progress so far

- Worked through the new clemcore environment architecture (agent-environment interface in PettingZoo and OpenEnv)
- (The framework is still evolving, requiring extra effort during initial integration)
- Implemented a minimal LangChain-based agent (verified end-to-end compatibility with the updated clemcore setup)